

CREATE TABLE, INSERT, SELECT and WHERE Queries in SQL

Introduction

SQL (Structured Query Language) is the standard language used to interact with databases. Before storing and retrieving data, we need to create tables and insert records into them. Once data is stored, we can use SELECT and WHERE queries to retrieve specific information.

In this lesson, we will learn:

- How to create tables using CREATE TABLE
 - How to add records using INSERT
 - How to retrieve data using SELECT
 - How to filter records using WHERE
-

1. CREATE TABLE Query

What is a Table?

A table is a collection of related data organized into rows and columns.

Example: Student Table

Student_ID Name Age Course

101 Rahul 20 AI

102 Priya 21 Data Science

Here:

- Columns represent attributes (Student_ID, Name, Age, Course)
 - Rows represent records
-

Syntax of CREATE TABLE

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    column3 datatype  
);
```

Example 1: Creating a Student Table

```
CREATE TABLE Students (  
  Student_ID INT,  
  Name VARCHAR(50),  
  Age INT,  
  Course VARCHAR(50)  
);
```

Explanation

Component	Meaning
-----------	---------

CREATE TABLE Creates a new table

Students Table name

INT Integer values

VARCHAR(50) Text up to 50 characters

Common SQL Data Types

Numeric Data Types

Data Type	Description
-----------	-------------

INT Whole numbers

FLOAT Decimal values

DOUBLE Large decimal values

Example:

Price FLOAT

Character Data Types

Data Type	Description
-----------	-------------

CHAR(n) Fixed-length text

VARCHAR(n) Variable-length text

Example:

Name VARCHAR(100)

Date and Time Data Types

Data Type Description

DATE Stores date

TIME Stores time

DATETIME Stores date and time

Example:

Joining_Date DATE

Using Constraints

Constraints define rules for data.

PRIMARY KEY

Ensures unique values.

```
CREATE TABLE Students (  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT  
);
```

NOT NULL

Prevents empty values.

```
CREATE TABLE Students (  
    Student_ID INT,  
    Name VARCHAR(50) NOT NULL  
);
```

UNIQUE

Prevents duplicate values.

```
CREATE TABLE Students (  
    Email VARCHAR(100) UNIQUE  
);
```

DEFAULT

Assigns a default value.

```
CREATE TABLE Students (  
    Status VARCHAR(20) DEFAULT 'Active'  
);
```

Complete Example

```
CREATE TABLE Employees (  
    Employee_ID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Department VARCHAR(50),  
    Salary FLOAT,  
    Joining_Date DATE  
);
```

2. INSERT Query

What is INSERT?

The INSERT statement is used to add data into a table.

Syntax

```
INSERT INTO table_name  
VALUES(value1, value2, value3);
```

Example

```
INSERT INTO Students  
VALUES(101, 'Rahul', 20, 'AI');
```

Result:

Student_ID	Name	Age	Course
------------	------	-----	--------

101	Rahul	20	AI
-----	-------	----	----

Inserting Multiple Records

```
INSERT INTO Students  
VALUES  
(102, 'Priya', 21, 'Data Science'),
```

```
(103, 'Aman', 22, 'Machine Learning'),  
(104, 'Neha', 20, 'Cyber Security');
```

Inserting into Specific Columns

```
INSERT INTO Students (Student_ID, Name, Course)  
VALUES (105, 'Karan', 'Python');
```

This leaves Age as NULL.

Employee Example

```
INSERT INTO Employees  
VALUES  
(1, 'John', 'HR', 35000, '2024-01-10'),  
(2, 'Emma', 'IT', 55000, '2023-12-15'),  
(3, 'David', 'Finance', 50000, '2024-03-01');
```

3. SELECT Query

What is SELECT?

The SELECT statement is used to retrieve data from a table.

Syntax

```
SELECT column_name  
FROM table_name;
```

Selecting All Columns

```
SELECT * FROM Students;
```

Output:

Student_ID Name Age Course

```
101      Rahul 20  AI  
102      Priya 21  Data Science
```

* means all columns.

Selecting Specific Columns

```
SELECT Name, Course  
FROM Students;
```

Output:

Name	Course
Rahul	AI
Priya	Data Science

Example

```
SELECT Employee_ID, Name  
FROM Employees;
```

Selecting Unique Values

```
SELECT DISTINCT Course  
FROM Students;
```

Output:

Course
AI
Data Science
Machine Learning

Column Aliases

Aliases provide temporary names.

```
SELECT Name AS Student_Name  
FROM Students;
```

Output:

Student_Name
Rahul
Priya

4. WHERE Query

What is WHERE?

The WHERE clause filters records based on a condition.

Without WHERE:

- All rows are returned.

With WHERE:

- Only matching rows are returned.
-

Syntax

```
SELECT column_name  
FROM table_name  
WHERE condition;
```

Example 1

```
SELECT *  
FROM Students  
WHERE Age = 20;
```

Output:

Student_ID	Name	Age	Course
101	Rahul	20	AI
104	Neha	20	Cyber Security

Comparison Operators

Operator Meaning

=	Equal to
>	Greater than
<	Less than
>=	Greater than or equal

Operator Meaning

<= Less than or equal

!= or <> Not equal

Examples

Greater Than

```
SELECT *  
FROM Employees  
WHERE Salary > 50000;
```

Less Than

```
SELECT *  
FROM Students  
WHERE Age < 21;
```

Not Equal To

```
SELECT *  
FROM Students  
WHERE Course <> 'AI';
```

Logical Operators in WHERE

AND Operator

Both conditions must be true.

```
SELECT *  
FROM Employees  
WHERE Department = 'IT'  
AND Salary > 50000;
```

OR Operator

At least one condition must be true.

```
SELECT *  
FROM Students
```


WHERE Age = 20
OR Course = 'AI';

NOT Operator

Reverses the condition.

```
SELECT *  
FROM Students  
WHERE NOT Course = 'AI';
```

BETWEEN Operator

Used to find values within a range.

```
SELECT *  
FROM Employees  
WHERE Salary BETWEEN 30000 AND 60000;
```

IN Operator

Checks multiple values.

```
SELECT *  
FROM Students  
WHERE Course IN ('AI', 'Python', 'Data Science');
```

LIKE Operator

Used for pattern matching.

Names Starting with 'R'

```
SELECT *  
FROM Students  
WHERE Name LIKE 'R%';
```

Examples:

- Rahul
- Rohan
- Raj

Names Ending with 'a'

```
SELECT *  
FROM Students  
WHERE Name LIKE '%a';
```

Examples:

- Priya
- Ananya

Names Containing 'an'

```
SELECT *  
FROM Students  
WHERE Name LIKE '%an%';
```

Examples:

- Ananya
- Karan

IS NULL and IS NOT NULL

Find Empty Values

```
SELECT *  
FROM Students  
WHERE Age IS NULL;
```

Find Non-Empty Values

```
SELECT *  
FROM Students  
WHERE Age IS NOT NULL;
```

Complete Example

Create Table

```
CREATE TABLE Students (  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(50),
```

```
Age INT,  
Course VARCHAR(50)  
);
```

Insert Data

```
INSERT INTO Students  
VALUES  
(101, 'Rahul', 20, 'AI'),  
(102, 'Priya', 21, 'Data Science'),  
(103, 'Aman', 22, 'Machine Learning'),  
(104, 'Neha', 20, 'Cyber Security');
```

Retrieve All Data

```
SELECT * FROM Students;
```

Retrieve Specific Columns

```
SELECT Name, Course  
FROM Students;
```

Apply Filter

```
SELECT *  
FROM Students  
WHERE Age = 20;
```

Best Practices

1. Use meaningful table and column names.
 2. Always define a Primary Key whenever possible.
 3. Use appropriate data types.
 4. Insert only valid and clean data.
 5. Use WHERE conditions carefully before retrieving large datasets.
 6. Prefer selecting specific columns instead of SELECT * in production environments.
 7. Maintain consistent naming conventions.
-

Summary

- CREATE TABLE is used to create new database tables.
- Data types define the type of information stored in each column.
- Constraints help maintain data integrity.
- INSERT INTO adds records to a table.
- SELECT retrieves data from tables.
- WHERE filters records based on conditions.
- Operators like AND, OR, IN, BETWEEN, and LIKE help create powerful queries.
- Proper table design and query writing are essential for efficient database management.

Key SQL Commands Learned

CREATE TABLE

INSERT INTO

SELECT

WHERE

DISTINCT

AND

OR

NOT

BETWEEN

IN

LIKE

IS NULL

IS NOT NULL